

EvidenceCheck Benchmark Runner

v2.1

Alejandro Peralta

Scientific claim verification with PubMed + AI consensus analysis

What is this?

A Python script that runs claims against your EvidenceCheck pipeline (n8n) and compares results against expected values. It gives you data to know whether a pipeline change improves or regresses the system, instead of guessing.

Files

File	Claims	Purpose
benchmark_core.csv	25	High-consensus biomedical claims. Used to evaluate baseline performance and system stability.
benchmark_niche.csv	15	High-uncertainty biomedical claims. The domain where EvidenceCheck AI demonstrates real value over a standalone LLM.
benchmark_runner.py	—	Execution and reporting script.

Installation

No external dependencies required (Python 3.8+ only):

```
python benchmark_runner.py --help
```

Usage

1. Establish baseline (first time)

```
python benchmark_runner.py \
  --webhook https://n8n.tudominio.com/webhook/evidence-check-submit \
  --result-url https://n8n.tudominio.com/webhook/evidence-check-result \
  --claims benchmark_core.csv \
  --output baseline.json
```

2. After each pipeline change

```
python benchmark_runner.py \  
  --webhook https://n8n.tudominio.com/webhook/evidence-check-submit \  
  --result-url https://n8n.tudominio.com/webhook/evidence-check-result \  
  --claims benchmark_core.csv \  
  --baseline baseline.json \  
  --output results_v2.json
```

3. Advanced biomedical benchmark (real pipeline value)

```
python benchmark_runner.py \  
  --webhook ... \  
  --result-url ... \  
  --claims benchmark_niche.csv \  
  --output niche_results.json
```

4. Custom score threshold

Default: 80% of claims must pass for exit code 0. Lower it during active development:

```
python benchmark_runner.py ... --min-score 0.70
```

5. Validate CSV without executing

```
python benchmark_runner.py --claims benchmark_core.csv --dry-run
```

Output

```
SCORE GLOBAL: 17/25 (68.0%)  
Passed: 17  
Failed: 8  
  
BY DIMENSION:  
Verdict      20/25 correct (80%)  
Confidence   18/25 correct (72%)  
Consensus    19/25 correct (76%)  
  
FAILURE PATTERNS DETECTED:  
MIXED consensus interpreted as MODERATE      5 case(s)  
EXAGGERATED claim marked as FALSE            2 case(s)  
Pipeline timeout (infrastructure)            1 case(s)  
  
REGRESSIONS (1):  
El café causa cáncer  
  Before: FALSE/MIXED  
  Now:    FALSE/STRONG  
  Reason: Expected consensus MIXED, received STRONG  
  
Score: 68.0% | Threshold: 80% | FAIL
```

Fixes in v2.1

1. Normalization with fallback (never loses information)

Previous issue: If Gemini returned an unmapped verdict (MOSTLY_TRUE, LIKELY_TRUE, or a typo), the benchmark returned None and marked failure.

Fix: If the value is not in the dictionary, the original value is returned (uppercased), not None:

```
# Before (bug):
return VERDICT_CANONICAL.get(val)      # MOSTLY_TRUE → None

# Now (fixed):
return VERDICT_CANONICAL.get(val, val) # MOSTLY_TRUE → MOSTLY_TRUE
```

This allows:

- Seeing new verdicts Gemini invents
- Detecting typos without losing the original information
- Manually comparing edge cases

2. TIMEOUT as a separate pattern

Previous issue: All pipeline errors collapsed into "Pipeline error: ..." without distinguishing between:

- Science failure (pipeline responded but verdict was wrong)
- Infrastructure failure (n8n crashed, timeout, error 500)

Fix: TIMEOUT is now detected as a separate failure pattern:

```
FAILURE PATTERNS DETECTED:
Pipeline timeout (infrastructure)      2 case(s)
Pipeline error (not science)          1 case(s)
MIXED consensus interpreted as MODERATE 5 case(s)
```

This is critical when running 25–50 benchmarks consecutively and n8n may saturate.

3. raw_result stored in output

Previous issue: If a claim passed yesterday and failed today, you only had the final verdict. You could not compare what changed in the pipeline.

Fix: Each result now includes `raw_result` with the full endpoint response:

```
{
  "claim_id": "abc-123",
  "actual_verdict": "EXAGERADO",
  "actual_consensus": "MIXTO",
  "raw_result": {
    "job_id": "job_abc123",
    "claim": "El azúcar causa cáncer",
    "verdict": "EXAGGERATED",
    "confidence": "MODERATE",
```

```
    "consensus": "MIXED",
    "articles": [{"pmid": "12345", "relation_to_claim": "SUPPORTS"}],
    "detailed_analysis": "...",
    "evidence_consensus": {...},
    "directionality": {...}
  }
}
```

For debugging: compare `results[16].raw_result` from yesterday vs today.

Fixes in v2.0 (previous)

Unique bidirectional normalization (canonical)

Everything is normalized to a single internal format before comparison:

```
CSV:      "MIXTO" → canonical: "MIXTO"
Pipeline: "MIXED" → canonical: "MIXTO"
Comparison: "MIXTO" == "MIXTO" ✓
```

papers_used corrected

Uses a heuristic based on `relation_to_claim`:

- If the article has `relation_to_claim` and it is not "UNCLEAR" → counts as used
- If it has no `relation_to_claim` → assumes all returned articles were used

Configurable exit code

`--min-score 0.80` (default). Pipeline passes if global score `>=` threshold.

```
24/25 = 96% | threshold 80% → exit 0 ✓
18/25 = 72% | threshold 80% → exit 1 ✗
18/25 = 72% | threshold 70% → exit 0 ✓
```

Advanced Biomedical Benchmark

The 25 claims in `benchmark_core.csv` are high-consensus biomedical claims used to evaluate baseline performance and system stability. They represent well-established scientific knowledge (tobacco → cancer, UV → melanoma, vaccines →/autism).

The 15 claims in `benchmark_niche.csv` are high-uncertainty biomedical claims — the domain where EvidenceCheck AI adds real value over a standalone LLM:

Claim	Why it is difficult	Expected
Occupational BPA exposure and T2D	Meta-analysis 2024, niche	PARTIALLY_TRUE

ctDNA predicts HCC recurrence	Recent biomarker	PARTIALLY_TRUE
NAD+ improves mitochondrial function	Hype vs evidence	EXAGGERATED
Semaglutide reduces dementia risk in T2D	2024 data, evolving	PARTIALLY_TRUE
SGLT2 inhibitors prevent HF in CKD without diabetes	DAPA-CKD/EMPA-KIDNEY	PARTIALLY_TRUE

Real value metric: If EvidenceCheck AI + PubMed scores 10/15 on advanced claims while a standalone Gemini model scores 4/15, the added value of the retrieval and evidence synthesis pipeline is demonstrated.

Failure Patterns

Pattern	Meaning	Suggested action
TIMEOUT	n8n did not respond within time limit	Check server load, increase timeout
PIPELINE_ERROR	Infrastructure error (not science)	Check n8n logs, connectivity
MIXED_vs_MODERATE	MIXED consensus interpreted as MODERATE	Review weighted consensus logic
EXAGERADO_vs_FALSO	EXAGGERATED claim marked as FALSE	Review verdict rules for harm claims
FALSO_vs_EXAGERADO	FALSE claim marked as EXAGGERATED	Review EXAGGERATED vs FALSE threshold
UNEXPECTED_INSUFFICIENT	Unexpected EVIDENCIA_INSUFICIENTE	Review retrieval / query generation
MODERATE_vs_HIGH_CONFIDENCE	MODERATE confidence marked as HIGH	Review bias/heterogeneity penalties
STRONG_vs_MIXED_CONSENSUS	STRONG consensus interpreted as MIXED	Review institutional consensus overrides
ANALYSIS_ERROR	Error in Gemini analysis	Review prompt or response format

Planned CI/CD Integration

Future versions of EvidenceCheck AI will include automatic benchmark execution through GitHub Actions.

Planned workflow: [.github/workflows/benchmark.yml](#)

Features:

- Run benchmark_core.csv automatically on every push
- Detect regressions automatically before merging changes

- Upload benchmark artifacts for historical comparison
- Fail CI if global score drops below configured threshold

CSV Format

```
id,claim,category,expected_verdict,expected_confidence,expected_consensus,causal_claim,expected_reasoning,common_failure,allowed_verdicts,allowed_confidence,allowed_consensus
```

```
abc-123,"El azúcar causa cáncer",oncology,EXAGERADO,MODERATE,MIXED,false,"Claim demasiado general...", "Generalizar evidencia parcial...",EXAGERADO|FALSO,MODERATE,MIXED|MODERATE
```

Field	Description
id	Unique UUID for the claim
claim	Claim text
category	Thematic category
expected_verdict	Expected verdict (Spanish or English, normalized)
expected_confidence	Expected confidence (HIGH/MODERATE/LOW)
expected_consensus	Expected consensus (STRONG/MODERATE/WEAK/MIXED/INSUFFICIENT)
causal_claim	Does the claim imply causality? (true/false)
expected_reasoning	Expected reasoning (for manual validation)
common_failure	Typical failure mode for this claim
allowed_verdicts	Acceptable verdicts separated by (for borderline cases)
allowed_confidence	Acceptable confidence values separated by
allowed_consensus	Acceptable consensus values separated by

Recommended Workflow

Week 1 — Baseline

- Run `benchmark_core.csv` → `baseline.json`
- Analyze the most common failure patterns
- Prioritize fixes by impact (those affecting the most claims)

Week 2 — Fix + Test

- Make ONE change in the pipeline (e.g. simplify prompt)
- Re-run with `--baseline baseline.json`
- If no regressions and improvements detected → merge
- If regressions appear → revert or adjust

Week 3+ — Advanced Benchmark

- Run `benchmark_niche.csv`
- Compare against standalone Gemini (manually)
- Document differences
- If EvidenceCheck wins significantly → value is proven

Debugging with `raw_result`

When a claim fails unexpectedly:

```
# 1. Run and save results
python benchmark_runner.py ... --output results.json

# 2. Inspect the raw_result of the failed claim
python -c "
import json
with open('results.json') as f:
    data = json.load(f)
for r in data['results']:
    if not r['overall_pass']:
        print(f'Claim: {r["claim"]}')
        print(f'Expected:
{r["expected_verdict"]}/{r["expected_confidence"]}/{r["expected_consensus"]}
}')
        print(f'Actual:
{r["actual_verdict"]}/{r["actual_confidence"]}/{r["actual_consensus"]}')
        print(json.dumps(r['raw_result'], indent=2, ensure_ascii=False)
[:2000])
        break
"
```

This shows exactly what the pipeline returned, including:

- Which articles were found
- What scores were assigned to each
- What consensus was calculated before Gemini overrode it
- Whether an institutional consensus override was applied